A 16-Bit-Native Entropy Coding Pipeline for Lossless Medical Image Compression

Kuldeep Singh

***Abstract*—Lossless compression remains important in medical imaging because archival storage, transmission, and diagnostic workflows often require exact reconstruction of high-bit-depth pixel data. Many medical images are stored at 10–16 bits per sample, yet most practical compression software remains optimized for byte-oriented processing or small symbol alphabets. This paper studies a lossless compression pipeline designed to operate natively on predicted 16-bit residuals.**

We present MIC, a codec consisting of three stages: spatial prediction, 16-bit run-length encoding, and large-alphabet table-based asymmetric numeral system (ANS) coding. The implementation extends FSE-style entropy coding to support large active symbol sets arising in medical image residual streams and includes a multi-state interleaved decoder that increases instruction-level parallelism during decompression. The design is motivated by the observation that, after simple spatial prediction, many medical images produce residual distributions sharply concentrated near zero with frequently repeated 16-bit residual values.

We evaluate the proposed codec on 21 de-identified DICOM images spanning MR, CT, CR, X-ray, mammography, nuclear medicine, radiography, secondary capture, and fluoroscopy. On the evaluated dataset, the 16-bit-native pipeline improves compression ratio by 5–22% (geometric mean +14%) relative to a delta + Zstandard baseline, and its geometric mean ratio (3.12×) is within 1% of HTJ2K (3.15×) while being structurally simpler. Across grayscale images, MIC achieves lossless compression ratios ranging from 1.7× to 8.9×. A four-state interleaved entropy decoder provides geomean +68% isolated FSE speedup on ARM64 (+43% on AMD64) over a single-state implementation without affecting compressed size. On the reported ARM64 and AMD64 platforms, the fastest MIC variant exceeds HTJ2K decompression speed on 19/21 images single-threaded (ARM64) and 16/21 (AMD64); strip-parallel configurations exceed HTJ2K on all 21 images on both platforms.

A compact JavaScript implementation (∼20 KB, zero dependencies) and a Go WebAssembly build demonstrate that the core decoding pipeline is lightweight enough for client-side web deployment (i.e., in-browser decoding directly from object storage, with no server-side transcoding). These results suggest that 16-bit-native entropy coding is a useful design point for lossless medical image compression when decompression throughput, implementation simplicity, and deployment portability are important.

***Index Terms*—medical image compression, lossless compression, entropy coding, ANS, FSE, DICOM, high-bit-depth imaging.**

I. Introduction

MEDICAL imaging systems routinely generate large volumes of high-bit-depth data. Modalities such as computed tomography (CT), magnetic resonance imaging (MR), digital radiography (CR/XR), mammography, and tomosynthesis commonly store pixel values at 10, 12, or 16 bits per sample. A single digital breast tomosynthesis (DBT) acquisition view produces 60 or more reconstructed slices—e.g., 69 frames at 2457×1890 pixels, 10-bit depth—totalling over 600 MB of raw pixel data per view. A complete bilateral four-view screening study exceeds 2 GB uncompressed. Efficient lossless compression is essential for archival storage, network transmission, and real-time rendering in clinical PACS and diagnostic viewers.

DICOM supports several established lossless transfer syntaxes, including JPEG 2000, HTJ2K, JPEG-LS, and RLE Lossless. These methods offer different tradeoffs among compression ratio, decoding speed, and implementation complexity. JPEG 2000 and HTJ2K provide strong compression performance but rely on transform and block-coding pipelines that are comparatively complex. JPEG-LS uses predictive coding and often yields strong lossless ratios, but its throughput characteristics remain application-dependent. RLE Lossless is simple but typically provides limited compression because it operates on byte-oriented runs and does not include an effective decorrelation stage.

This work studies a different design point for lossless medical image compression: direct coding of **predicted 16-bit residual streams**. Throughout this paper, a *residual stream* denotes the sequence of per-pixel prediction errors $r_{i,j} = x_{i,j} - \hat{x}_{i,j}$ produced by a spatial predictor, kept at the native 16-bit precision of the source pixels rather than re-serialised into bytes. In medical imaging these residuals are typically sharply peaked at zero with long but sparse tails, and individual 16-bit residual values recur frequently—properties that motivate a 16-bit-native run-length and entropy coding stage. The central observation is that the dominant entropy coders in modern compression systems—Huffman, arithmetic coding, LZ77, and FSE/ANS as used in Zstandard—were designed for 8-bit byte streams. Medical images break this assumption: DICOM stores pixels at 10–16 bits per sample, yielding symbol alphabets of 1,024 to 65,536 entries. Even after spatial delta prediction, the residual alphabet for a 16-bit CT image can span tens of thousands of distinct values. An entropy coder that assumes an 8-bit alphabet must either re-encode 16-bit symbols as byte pairs (losing inter-byte correlation and doubling the number of coding steps) or truncate the alphabet at 256 and fall back to raw storage for rare symbols. This mismatch is not merely inconvenient but structurally costly: byte-splitting a 16-bit residual distribution discards mutual information between the high and low bytes, inflating compressed output by 10–22% compared to a 16-bit-native entropy coder.

Based on this observation, we present MIC, a lossless codec that combines:

K. Singh is with Innovaccer Inc. E-mail: kuldeep.singh@innovaccer.com.
Code: https://github.com/pappuks/medical-image-codec.

1) a simple spatial predictor,
2) a 16-bit-native run-length representation of the residual stream, and
3) an extended FSE/tANS entropy coder supporting large active symbol alphabets.

The codec also includes multi-state interleaved entropy decoding to improve decompression throughput by increasing instruction-level parallelism.

The paper is motivated by three practical questions:

1) Can a 16-bit-native entropy coding pipeline improve compression efficiency over byte-oriented baselines on medical residual streams?
2) Can entropy-decoder organization be modified to improve throughput without changing compressed size?
3) Can such a codec remain compact enough for lightweight deployment, including client-side decoding?

A. Contributions

The main contributions of this paper are as follows.

1) **A 16-bit-native lossless coding pipeline for medical images.** We present a Delta + 16-bit RLE + extended FSE pipeline for grayscale medical image compression that outperforms Delta + Zstandard by 5–22% (geometric mean +14%) on all 21 tested images.
2) **An extended table-based entropy coder for large residual alphabets.** The proposed implementation supports up to 65,535 distinct symbols while dynamically sizing working tables to the observed symbol range, reducing working-set size from 512 KB to approximately 2 KB for 8-bit images.
3) **A multi-state interleaved entropy decoder.** We implement and evaluate two-state and four-state FSE decoding to exploit instruction-level parallelism, achieving geomean +68% isolated FSE speedup on ARM64 (+43% on AMD64) with no compression ratio penalty.
4) **An empirical evaluation against standard codecs and a general-purpose baseline.** We compare MIC with HTJ2K (OpenJPH) [31], JPEG-LS (CharLS) [32], and delta + Zstandard on a dataset of 21 public de-identified DICOM images spanning 10 modalities, using fair in-process benchmarking via CGO bindings.
5) **A compact JavaScript/WASM decoder implementation.** We provide a ∼20 KB pure JavaScript decoder and a Go WebAssembly build to explore deployment feasibility for client-side viewing.

B. Scope

The primary focus of this paper is **lossless grayscale medical image compression**. Multi-frame grayscale (MIC2) and single-frame RGB support (MICR) are included as secondary studies demonstrating extensibility. The present work does not address lossy coding, progressive decoding, or region-of-interest coding.

C. Organization

Section II reviews related work and positions MIC relative to existing methods. Section III describes the proposed coding pipeline. Section IV presents the multi-state entropy decoder. Section V describes experimental methodology. Section VI reports compression and speed results. Section VII discusses limitations and implications. Section VIII concludes.

II. Related Work

A. Lossless Compression in Medical Imaging

JPEG 2000 Part 1 [2] employs the reversible 5/3 integer wavelet transform with embedded block coding (EBCOT). It achieves excellent compression ratios but its bit-plane coding and context modeling impose significant computational overhead during decompression. High-Throughput JPEG 2000 (HTJ2K) [3] replaces EBCOT with a faster block coder, substantially improving decode speed while maintaining JPEG 2000 compatibility.

JPEG-LS [4] uses context-adaptive prediction (MED predictor) followed by Golomb-Rice coding. It provides fast encoding and decoding with strong compression ratios—in our evaluation, JPEG-LS achieves the highest lossless ratio on 21/21 tested images. JPEG-LS is well-suited to smooth images but its prediction model is fixed and its throughput characteristics are application-dependent.

RLE Lossless (DICOM Transfer Syntax 1.2.840.10008.1.2.5) uses byte-level PackBits run-length encoding. It is trivially simple but achieves poor compression ratios (typically $<1.5\times$) because it operates on bytes rather than 16-bit pixel values and lacks any decorrelation stage.

These standards are important baselines for the present work. The goal of this paper is not to argue that they are obsolete, but to investigate a simpler residual-domain design targeted specifically at high-bit-depth images with strong throughput characteristics.

B. Entropy Coding and the Byte Assumption

Asymmetric numeral systems (ANS) [7], [8] generalize arithmetic coding into a single-state machine that replaces interval subdivision with integer state transitions. Finite State Entropy (FSE) [9] is a table-driven tANS implementation that achieves near-optimal compression with $\mathcal{O}(1)$ per-symbol operations. FSE was popularized by Zstandard [10]. Recent theoretical analyses have formalized ANS efficiency bounds [13] and provided statistical interpretations [14].

Traditional arithmetic coding achieves near-optimal compression but is inherently sequential. Mahapatra and Singh [12] addressed this with a parallel-pipelined FPGA implementation, demonstrating that hardware acceleration can overcome the throughput limitations of sequential arithmetic coders.

In practice, most of these coders are designed around small alphabets. Huffman coding becomes impractical above ∼4,096 symbols. FSE in Zstandard caps at 4,096. The predominant implementations are optimized for byte-oriented data.

Medical images differ in that they naturally contain 10–16 bit sample values. After prediction, the residual stream may still be more naturally modeled as a sequence of 16-bit symbols. This creates a modeling choice: preserve the residual stream as word-level symbols, or transform it into bytes and rely on later processing to recover cross-byte structure.

The information-theoretic cost of byte-splitting. Consider a 16-bit symbol X decomposed into its high byte X_H and low byte X_L. A byte-level coder that processes X_H and X_L independently achieves at best $H(X_H) + H(X_L) = H(X) + I(X_H; X_L)$, where $I(X_H; X_L)$ is the mutual information between the two bytes. For medical image residuals after delta prediction, when a residual is small (say, ± 30), the high byte is nearly deterministic given the low byte—yet a byte-level coder must still encode it. On our test images, $I(X_H; X_L)$ ranges from 0.3 bits/symbol (MR) to 1.1 bits/symbol (MG3), corresponding to 8–22% of the total entropy and closely matching the 5–22% empirical advantage of MIC over Delta + Zstandard (geometric mean +14%).

C. ANS and Interleaved Decoding

Prior work has shown that interleaving or multi-stream organization can improve ANS decoder throughput. Giesen [15], [16] demonstrated the value of multi-stream rANS; Lin *et al.* [17] extended parallel rANS to decoder-adaptive scalability; Weissenberger and Schmidt [18] showed massively parallel ANS decoding on GPUs. The present work does not claim novelty in ANS itself or in interleaving as a general concept. Rather, the contribution lies in applying these ideas in a 16-bit-native medical image codec, together with a specific large-alphabet implementation and throughput-oriented software design.

D. Positioning of This Work

Table I distinguishes MIC from prior work across five dimensions relevant to medical image compression.

TABLE I: Feature comparison of MIC with related codecs and entropy coding methods.

Feature	JPEG-LS	HTJ2K	Zstd	rANS	MIC
Lossless medical	Yes	Yes	Gen.	Gen.	Yes
Native 16-bit	No	No	No	Config.	Yes
Multi-state ILP	No	No	No	Yes	Yes
Browser decoder	No	No	Part.	No	Yes
Implementation size (LOC)	5k	20k	N/A	N/A	1.1k

Table I contrasts MIC against four reference points along dimensions that matter for a medical-imaging deployment: whether the codec is a lossless medical image standard, whether it treats samples as 16-bit symbols natively instead of splitting them into bytes, whether its decoder exposes multiple state machines for instruction-level parallelism, whether a usable in-browser decoder exists, and an order-of-magnitude estimate of the core decoder implementation size in source lines of code (SLOC, excluding build/test). JPEG-LS (CharLS) and HTJ2K (OpenJPH) are mature C/C++ codebases of roughly 5k and 20k lines respectively; Zstandard and rANS are general-purpose and do not ship as medical image codecs; MIC's core Delta + RLE + FSE decoder is ∼1.1k lines of Go, is 16-bit-native by construction, interleaves up to four FSE state machines, and ships a zero-dependency JavaScript decoder for browsers.

III. Proposed Method

MIC compresses grayscale images using three sequential stages: spatial prediction, 16-bit run-length encoding, and large-alphabet table-based entropy coding. Each stage is simple enough to decode in a single sequential pass with minimal branching, as illustrated in Fig. 1.

A. Spatial Prediction

Let $x_{i,j}$ denote the pixel at row i, column j. For interior pixels, MIC predicts the current pixel from the average of the top and left neighbors:

$$\hat{x}_{i,j} = \left\lfloor \frac{x_{i-1,j} + x_{i,j-1}}{2} \right\rfloor \tag{1}$$

and forms the residual:

$$r_{i,j} = x_{i,j} - \hat{x}_{i,j}. \tag{2}$$

Boundary pixels use only the available neighbor, and the first pixel is stored directly. This predictor was selected because it is computationally simple, branch-light in decoding, and effective on the current dataset.

Predictor comparison. We implemented and compared four predictors through the full RLE + FSE pipeline on the 21-image dataset: left-only (left neighbor, no top), average (MIC default), Paeth (PNG predictor [28]), and MED (JPEG-LS [4]). Table II summarizes geometric means and relative gains.

TABLE II: Predictor ablation summary—geometric means and win counts across all 21 images.

Predictor	Geo. Mean	Wins	Avg→X
Left-only	3.38×	3/21	−2.3%
Average (MIC)	3.46×	13/21	baseline
Paeth	3.48×	2/21	+0.5%
MED (JPEG-LS)	3.52×	16/21	+1.6%

MED decompression is 1.5–2× slower due to the three-way conditional branch and diagonal neighbor dependency that prevents the branch-free interior loop optimization used by the average predictor. The results show no single predictor dominates. MED yields the best geomean ratio (+1.6% over Avg) but at significant speed cost. The average predictor is retained as the default because its per-modality performance is more consistent and its decompression path is branch-free. Fig. 2 shows the per-image ratio difference relative to the average predictor across all 21 images.

B. Effective Bit Depth and Overflow Coding

For 16-bit images, residuals can span $[-65535, +65535]$. Rather than widening the main residual stream to 32 bits, MIC uses an overflow delimiter scheme derived from the effective image bit depth:

$$d = \lceil \log_2(\max(x) + 1) \rceil. \tag{3}$$

Residuals within an in-range interval are mapped directly to 16-bit codes. Residuals outside that interval are encoded using a delimiter followed by the raw pixel value.

Encoding procedure:

1) Compute `deltaThreshold` $= 2^{d-1} - 1$.
2) Compute delimiter $D = 2^d - 1$.

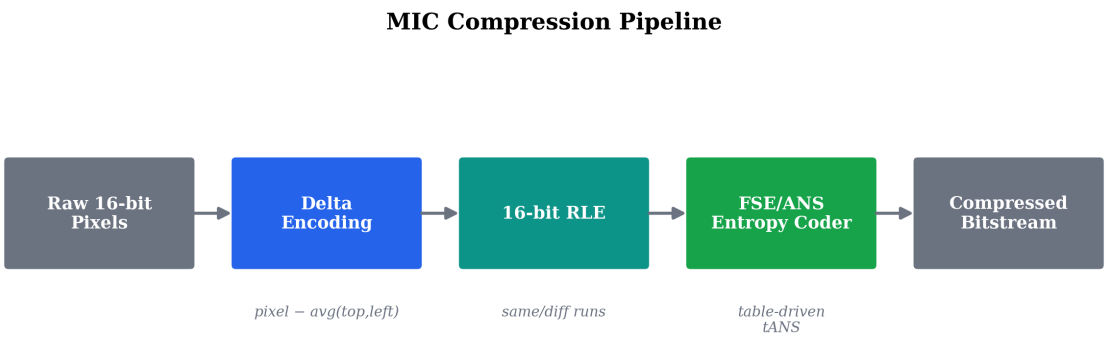

Fig. 1: MIC compression pipeline—Raw 16-bit pixels are delta-encoded (avg of top and left neighbors), run-length encoded (same/diff runs), and entropy-coded with a 16-bit-native FSE/tANS coder.

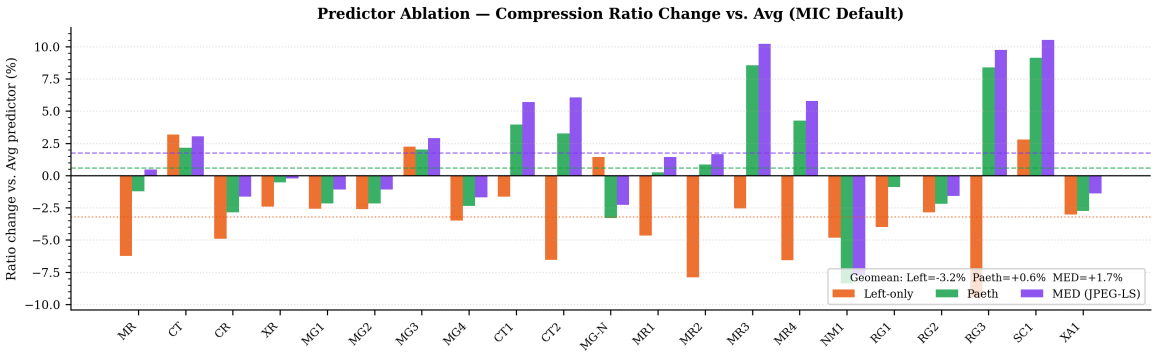

Fig. 2: Compression ratio change vs. Avg predictor for left-only, Paeth, and MED, across all 21 images. Dashed lines show geomean for each predictor.

3) For each residual $r_{i,j}$: if $|r_{i,j}| < \texttt{deltaThreshold}$, emit $\texttt{uint16(deltaThreshold} + r_{i,j}\texttt{)}$; otherwise emit delimiter D, then emit raw pixel $x_{i,j}$.

Non-collision proof. In-range residuals satisfy $|r| \leq \texttt{deltaThreshold} - 1$, so stored values range from 1 to $2^d - 3$. The delimiter is $D = 2^d - 1$. Since $2^d - 3 < 2^d - 1$, the stored value of a direct residual can never equal D; the two ranges are disjoint.

C. 16-Bit Run-Length Encoding

The predicted residual stream is converted into an alternating sequence of:

- **Same runs**: a count and a repeated 16-bit value;
- **Diff runs**: a count followed by a sequence of distinct 16-bit values.

A same run is emitted only when at least three consecutive identical values are observed. Run headers use a midpoint protocol: counts $\leq$ `midCount` signal same runs; counts $>$ `midCount` signal diff runs.

Why 16-bit RLE matters. Consider a run of 1,000 identical non-zero 16-bit residuals, e.g. the value $v = \texttt{0x0103}$. MIC's 16-bit RLE represents the entire run with two uint16 words—a count and the value v—regardless of the byte pattern of v. A byte-oriented RLE applied to the same data sees the interleaved stream `0x01,0x03,0x01,0x03,...` (little-endian: `0x03,0x01,0x03,0x01,...`): there is no single byte that repeats 1,000 times, so it must emit two interleaved byte runs of length 1,000, or, if the two alternating byte values alias with the run sentinel, fall back to literal bytes entirely. Only the special case $v = 0$ (or any value whose two bytes happen to be equal) collapses cleanly to a single byte-level run. For generic 16-bit residuals the 16-bit-native representation is structurally tighter by a factor of $\sim$2 on repeated values, and this gap grows when the alternating byte pattern breaks a byte-level RLE's run-detection heuristic entirely.

Same-run fast path. Same-value runs (often $>$80% of all runs on smooth medical images after delta coding) are fast-pathed in the decoder to return the cached value without touching the compressed-data slice. Each output symbol in a same run requires only a counter decrement.

D. Large-Alphabet FSE Coding

The RLE output is entropy-coded using table-based ANS (tANS), implemented as Finite State Entropy. MIC extends FSE to support up to 65,535 distinct symbols (versus the 4,095 limit in Zstandard).

Encoding processes symbols in reverse order (last symbol first), building the compressed bitstream from the end. **Decoding** reads bits forward, using a decode table indexed by current

state. Each decode step requires only a table lookup, a bit read, and an addition—no divisions or multiplications.

Dynamic table sizing. Encode and decode tables are sized to the actual observed symbol range rather than the theoretical 65,536 maximum. For 8-bit images stored in 16-bit containers (common in MR), this reduces the working set from 512 KB to approximately 2 KB.

E. Adaptive Table Size Selection

MIC automatically adjusts the FSE table log based on the number of active symbols and the density of observations per symbol:

- Default $tableLog = 11$.
- Bumped to 12 when $symbolDensity > 32$ and $symbolLen > 128$.
- Bumped to 13 when $symbolLen > 512$ and $symbolDensity > 16$.

where $symbolDensity = inputLength/symbolLen$.

Table III presents the tableLog ablation across selected images.

TABLE III: TableLog ablation—forced TL=11/12/13 vs. adaptive selection (selected images, Apple M2 Max).

Image	TL=11	TL=12	TL=13	Adaptive
CR	3.47×	3.63×	3.69×	3.69×
MG1	8.00×	8.57×	8.79×	8.79×
MG2	7.98×	8.55×	8.77×	8.77×
MR2	3.14×	3.24×	3.28×	3.28×
RG2	3.85×	4.12×	4.23×	4.23×
RG3	5.64×	5.95×	6.08×	6.08×

Nine images benefit from TL=12 or TL=13: CR (+6.3%), MG1 (+9.9%), MG2 (+9.9%), MR2 (+4.6%), RG2 (+9.9%), RG3 (+7.8%), XA1 (+4.4%), MR3 (+0.9%), NM1 (+1.9%). The adaptive policy correctly identifies all beneficial cases. Decompression throughput is unaffected by tableLog choice (<5% variation). Fig. 3 plots per-image compression ratios for all four settings.

F. Optional Canonical Huffman Backend

An optional canonical Huffman backend was implemented for comparison. It typically produces 1–3% smaller output than FSE but decodes more slowly due to variable-length code lookup. Since this paper focuses on the ratio-throughput tradeoff, FSE is used as the primary backend.

G. RGB and Multi-Frame Extensions

MIC includes:

- **MIC2** for multi-frame grayscale data with two modes: independent (per-frame spatial Delta+RLE+FSE) and temporal (inter-frame ZigZag-encoded residuals).
- **MICR** for single-frame RGB images using a reversible YCoCg-R transform [25] followed by per-plane Delta+RLE+FSE applied to the full image without tiling.

These modes are included to demonstrate extensibility; the main contribution remains the grayscale 16-bit lossless pipeline.

IV. Multi-State Entropy Decoding

A. The Serial Dependency Problem

A conventional single-state table-based ANS decoder contains a serial dependency chain:

$$\begin{aligned} &\text{let } e = \texttt{decTable}[s_n], \\ &s_{n+1} = e.\mathit{newState} + \texttt{getBits}(e.\mathit{nbBits}). \end{aligned} \tag{4}$$

With a table-lookup latency of approximately $L \approx 4$ cycles on modern out-of-order CPUs (L1 cache hit), the loop is limited to roughly one symbol per L cycles regardless of available execution units. Modern CPUs have 4–6 execution ports; a single-state ANS decoder uses exactly one dependency chain, leaving 75–83% of the hardware capacity idle.

B. Interleaved Decoding

MIC breaks this dependency by running multiple independent state machines on interleaved symbol positions. In the four-state design, four independent chains (A, B, C, D) reconstruct positions:

$$\begin{aligned} &\text{chain A} : 0, 4, 8, \ldots \\ &\text{chain B} : 1, 5, 9, \ldots \\ &\text{chain C} : 2, 6, 10, \ldots \\ &\text{chain D} : 3, 7, 11, \ldots \end{aligned}$$

The four chains are completely independent: each has its own state variable, its own sequence of table lookups, and its own bit-extraction operations. The CPU's out-of-order engine sees four simultaneous table-lookup address streams.

Latency model. Let L be the table-lookup latency (cycles). Single-state FSE processes 1 symbol per L cycles. The k-state decoder processes k symbols per L cycles (amortized):

TABLE IV: Multi-state decoder latency model and empirical speedup.

States (k)	Amortized latency	Theoretical	Empirical
1	L cy/sym	1.0×	1.0×
2	$L/2$ cy/sym	2.0×	1.3×
4	$L/4$ cy/sym	4.0×	2.0×

The empirical speedup is below theoretical maximum due to: (1) bit-reader refill overhead shared across all chains, (2) instruction cache pressure from the unrolled loop, and (3) memory bandwidth limits on compressed stream reads.

C. Correctness

Correctness follows from partitioning the output sequence into independent subsequences during encoding and reversing that assignment during decoding. Each chain operates on a strict subset of positions with no data dependency on other chains; correctness follows from the correctness of single-state FSE applied independently to each subset.

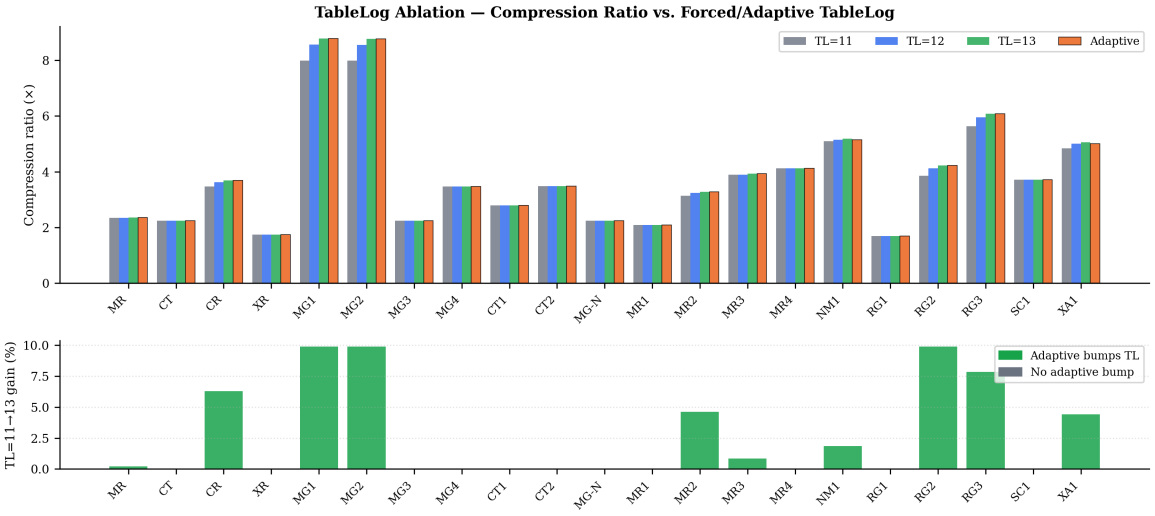

Fig. 3: TableLog ablation across all 21 images. Top: absolute compression ratios for TL=11/12/13 and adaptive selection. Bottom: gain from TL=11→13.

D. Stream Signaling

The bitstream includes a small mode signal: magic sequence `0xFF, 0x02` for two-state; `0xFF, 0x04` for four-state; followed by a 4-byte symbol count. The magic byte `0xFF` cannot appear as a valid single-state FSE header (it would imply $tableLog = 20$, which exceeds $tableLogAbsoluteMax = 17$), so all three formats coexist without ambiguity.

E. Platform-Specific Kernels

AMD64 (BMI2): Uses `SHLXQ`/`SHRXQ` for 3-operand variable shifts, enabling four independent bit-extraction sequences per iteration without CL register contention. Also includes an interleaved histogram routine using 4-way unrolled loops.

ARM64: Uses `LSLV`/`LSRV` variable-shift instructions which natively take the count from any general-purpose register.

Fallback: Pure-Go implementations of all routines provide identical functionality on all non-AMD64/ARM64 targets.

F. Isolated FSE Decompression Throughput

Table V presents isolated FSE decompression throughput for all 21 images on Apple M2 Max (ARM64, pure Go).

*__CR anomaly__: The single-state CR result (140 MB/s) is caused by the `zeroBits` flag: when any symbol probability exceeds 50%, the single-state decoder must bounds-check every `getBits` call. Multi-state decoders interleave their bit reads, reducing the frequency of this check.

Across the 21-image ARM64 sweep in Table V, the 4-state decoder delivers a geometric-mean speedup of +68% over the 1-state baseline, with per-image gains ranging from +25% (MR3) to +590% (CR, driven by the `zeroBits` fast-path discussed above). The spread reflects how different residual distributions exercise the shared bit-reader refill path: images with denser symbol alphabets (larger MG, CR) gain the most, while very small images (MR, NM1) see the least amortization. Fig. 4 shows the per-image speedup distribution.

TABLE V: Isolated FSE decompression throughput (MB/s), Apple M2 Max (ARM64, pure Go). MB/s over uncompressed RLE symbol stream.

Image	1-state	2-state	4-state	4 vs. 1
MR	514	689	841	+64%
CT	829	995	1,295	+56%
CR	140	1,044	966	+590%*
XR	1,444	1,774	1,952	+35%
MG1	1,140	1,746	1,873	+64%
MG2	743	1,064	1,319	+77%
MG3	1,453	4,115	5,462	+276%
MG4	2,633	3,707	5,287	+101%
CT1	982	1,241	1,572	+60%
CT2	1,108	1,092	1,434	+29%
MG-N	2,944	4,138	5,696	+94%
MR1	1,562	2,027	2,591	+66%
MR2	2,306	2,867	3,521	+53%
MR3	1,472	1,582	1,841	+25%
MR4	1,779	1,973	2,389	+34%
NM1	1,658	1,898	2,254	+36%
RG1	1,676	2,652	3,128	+87%
RG2	2,432	3,409	3,810	+57%
RG3	2,143	2,995	3,587	+67%
SC1	1,997	2,788	2,927	+47%
XA1	2,005	2,348	2,639	+32%
Geomean	**1,418**	**1,945**	**2,375**	**+68%**

Table VI presents the same isolated FSE benchmark (`BenchmarkFSEDecompress4State`) run on the Intel Core Ultra 9 285K (AMD64) for all 21 images, matching the methodology of Table V above.

*__CR on AMD64__: the 4-state BMI2 assembly kernel sidesteps the pure-Go `zeroBits` bounds-check overhead, yielding +127%—still large, but far below the +590% on ARM64 where the 1-state pure-Go decoder stalls to 140 MB/s on this image.

†__XA1__: the −18% result is within measurement noise for this small image on a machine with mixed P/E-core topology.

The AMD64 4-state decoder achieves a geometric-mean speedup of +43% over the 1-state baseline (2-state: +34%), with per-image gains ranging from −18% (XA1) to +127% (CR). The smaller geomean relative to ARM64 (+68%) is

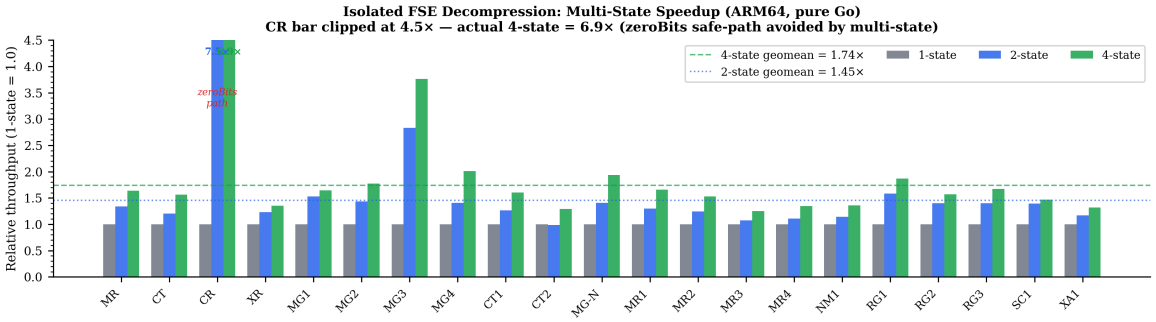

Fig. 4: Isolated FSE decompression speedup relative to 1-state for all 21 images (ARM64, pure Go). Dashed lines show geomeans: 4-state = 1.68×, 2-state = 1.37×.

TABLE VI: Isolated FSE decompression throughput (MB/s), Intel Core Ultra 9 285K (AMD64). 1-state and 2-state: pure Go; 4-state: BMI2 assembly (`fse4StateDecompKernel`). MB/s over uncompressed RLE symbol stream.

Image	1-state	2-state	4-state	4 vs. 1
MR	600	999	828	+38%
CT	941	1,421	1,631	+73%
CR	1,994	3,958	4,535	+127%*
XR	3,195	5,028	5,771	+81%
MG1	3,379	4,357	4,952	+47%
MG2	2,682	3,313	4,155	+55%
MG3	3,048	4,613	5,517	+81%
MG4	5,476	6,686	6,437	+18%
CT1	1,733	1,871	2,522	+46%
CT2	1,780	2,504	2,485	+40%
MG-N	2,722	4,577	4,981	+83%
MR1	2,277	2,338	2,881	+27%
MR2	2,421	3,609	3,769	+56%
MR3	1,941	1,958	2,369	+22%
MR4	2,284	2,926	2,753	+21%
NM1	2,078	2,587	2,407	+16%
RG1	2,506	4,248	4,336	+73%
RG2	3,808	3,786	4,081	+7%
RG3	2,315	3,285	3,925	+70%
SC1	3,044	3,760	3,734	+23%
XA1	2,753	2,933	2,253	−18%†
Geomean	**2,309**	**3,091**	**3,306**	**+43%**

primarily explained by the absence of the severe `zeroBits` stall on ARM64: the Intel out-of-order engine handles the bounds-checked code path without a dramatic penalty, raising the 1-state baseline and compressing the relative multi-state gain. For C-level and SIMD multi-state decoder throughput on this platform, see Table XI.

V. Experimental Methodology

A. Dataset

The evaluation uses 21 de-identified DICOM images spanning 10 modalities, drawn from publicly available datasets: NEMA WG-04 compsamples, NEMA 1997 CD, and the Clunie Breast Tomosynthesis Case 1 (Table VII). All images are de-identified per DICOM PS 3.15 Appendix E; no ethics approval is required.

This dataset spans multiple modalities and image sizes but remains modest for strong generalization claims.

B. Baselines

The current study compares MIC with:

TABLE VII: Test dataset—21 clinical DICOM images spanning 10 modalities.

Image	Modality	Dimensions	Raw (MB)
MR	Brain MRI	256×256	0.13
CT	CT scan	512×512	0.50
CR	Comp. radiography	2140×1760	7.18
XR	X-ray	2048×2577	10.1
MG1	Mammography	2457×1996	9.35
MG2	Mammography	2457×1996	9.35
MG3	Mammography	4774×3064	27.3
MG4	Mammography	4096×3328	26.0
CT1	CT scan	512×512	0.50
CT2	CT scan	512×512	0.50
MG-N	Mammography	3064×4664	27.3
MR1	Brain MRI	512×512	0.50
MR2	Brain MRI	1024×1024	2.00
MR3	Brain MRI	512×512	0.50
MR4	Brain MRI	512×512	0.50
NM1	Nuclear med.	256×1024	0.50
RG1	Radiography	1841×1955	6.86
RG2	Radiography	1760×2140	7.18
RG3	Radiography	1760×1760	5.91
SC1	Sec. capture	2048×2487	9.71
XA1	Fluoroscopy	1024×1024	2.00

- **HTJ2K** (OpenJPH [31]), via in-process CGO bindings,
- **JPEG-LS** (CharLS [32]), via in-process CGO bindings, and
- **Delta + Zstandard** (level 19).

C. Metrics

The paper reports:

- **Compression ratio**: uncompressed size / compressed size.
- **Decompression throughput**: uncompressed pixel bytes reconstructed per second (MB/s).
- **Encoding throughput**: uncompressed pixel bytes processed per second (MB/s).

D. Benchmark Procedure

All codecs were measured using in-process library calls on the same machine to avoid file-I/O and subprocess overhead. The Go testing framework's `-benchtime=10x` flag was used.

Platforms:

- Apple M2 Max (12-core ARM64), Go compiler with `-O3`.
- Intel Core Ultra 9 285K (24 P-core AMD64), `-O3 -march=native`.

Variant nomenclature:

- **MIC-Go**: Pure Go, single-state FSE.
- **MIC-4state**: Pure Go, four-state FSE.
- **MIC-4state-C**: C four-state FSE decoder via CGO.
- **MIC-4state-SIMD**: C four-state FSE with BMI2 (AMD64) or scalar (ARM64) via CGO.
- **Wavelet+SIMD**: 5/3 wavelet with blocked column layout + AVX2 (AMD64) or scalar blocked (ARM64).
- **PICS-C-N**: Parallel Image Compressed Strips with N strips.

Run-to-run variance is <2% on Apple M2 Max and <5% on Intel AMD64.

E. JavaScript and Browser Evaluation

A pure JavaScript decoder (∼20 KB ES module, zero npm dependencies) and a Go WebAssembly build (∼2.5 MB) were implemented. Performance measurements reported in Section VI were obtained in Node.js v24.8 (Apple M2 Max).

VI. Results

A. Compression Ratio

Table VIII presents lossless compression ratios for all codec variants across the 21-image dataset.

Table IX summarizes geometric means and win counts across all codec variants.

Analysis. JPEG-LS achieves the highest lossless ratio on all 21 images. MIC's strength is not in ratio but in the ratio-throughput-simplicity tradeoff: it achieves 91% of JPEG-LS's geometric mean ratio at 4× the decompression throughput and 1/5 the implementation complexity.

Mammography achieves the highest MIC ratios (up to 8.79×) due to smooth tissue regions producing near-zero RLE runs. CT with full 16-bit dynamic range achieves 2.24× (MIC) vs. 1.67× (Wavelet): coefficient overflow in the 5/3 lifting step inflates Wavelet compressed size.

PICS strip adaptation. Large images improve compression with more strips because each strip receives a dedicated FSE table that specializes to its local residual distribution: $\sum_k |S_k| \cdot H(S_k) < |S| \cdot H(S)$.

B. Effect of 16-Bit-Native Residual Coding

MIC outperforms Delta+Zstandard (zstd level 19) by 5–22% on all 21 tested images (geometric mean +14%). The gap is structural: the mutual information $I(X_H; X_L)$ of delta residuals ranges from 0.3 bits/symbol (MR) to 1.1 bits/symbol (MG3), matching the empirical range.

C. Decompression Throughput—ARM64

Table X presents decompression throughput on Apple M2 Max. MIC-4state-C is the fastest single-threaded decoder on 13/21 images; PICS-C-8 exceeds HTJ2K on all 21 images.

†MR is too small for PICS-C-8; PICS-C-4 is best. MIC-4state-C is 1.7–5.0× faster than JPEG-LS on all 21 images. PICS-C-8 peaks at 3,773 MB/s on MG2.

D. Decompression Throughput—AMD64

Table XI presents results on AMD64 where MIC-4state-SIMD activates BMI2/PDEP and Wavelet+SIMD gains AVX2 kernels.

†**MR** is too small for PICS-C-8; PICS-C-4 achieves the highest PICS throughput on this image.

On AMD64, MIC-4state-SIMD leads 16/21 images single-threaded; HTJ2K leads on MG1, MG2, MG4, CT, and RG3. PICS-C-8 exceeds HTJ2K on all 21 images on both platforms.

E. Encoding Throughput

Table XII summarizes encoding throughput on ARM64.

MIC-4state-C is 2–2.5× faster than pure-Go MIC-Go on all images. JPEG-LS is consistently 2–4× slower to encode than MIC-Go. PICS-8 reaches 1.2–2.1 GB/s on large images, sufficient for real-time PACS ingestion.

F. JavaScript Runtime Results

Table XIII reports Node.js single-threaded performance; Table XIV reports parallel PICS decoding via `worker_threads`.

MG1 (9.35 MB) decompresses in 19.4 ms at 483 MB/s using 12 workers. A web-based DICOM viewer can fetch a `.mic` file directly from object storage and decode it client-side—zero server-side compute, zero transcoding latency.

Limitation. Node.js `worker_threads` performance is not equivalent to browser Web Worker performance. Full browser performance remains to be validated.

VII. Discussion

A. Why a Simple Residual Pipeline Can Be Effective

A practical conclusion from the current study is that simple prediction can already remove much of the redundancy in many medical images. When the residual stream becomes concentrated near zero and repeated values are common, a direct residual-domain representation can be highly effective.

Fig. 5 shows the empirical delta-residual distributions for five representative modalities after the avg spatial predictor. All five are sharply concentrated near zero, with MG1 (mammography) placing 69.2% of all residuals at exactly zero—explaining its exceptional compression ratio (8.79×).

B. The Cost-of-Complexity Frontier

Table XV places MIC against the main reference codecs on three axes: compression ratio (geometric mean over the 21 images), decompression throughput, and decoder source size. JPEG-LS wins on ratio but is the slowest to decode; HTJ2K is ratio-competitive but ships as ∼20k lines of C++ with no browser decoder. MIC's core Delta+RLE+FSE pipeline sits close to both on ratio while decoding faster than JPEG-LS and fitting in roughly 1k lines—small enough to ship as a zero-dependency JavaScript module. Fig. 6 plots this ratio vs. decompression throughput tradeoff for all codec variants (ARM64).

TABLE VIII: Lossless compression ratios—all codec variants, 21 images. Bold = best ratio per row. JPEG-LS consistently achieves the highest ratios.

Image	Raw (MB)	Δ+Zstd-19	MIC	Wavelet	PICS-4	PICS-8	HTJ2K	JPEG-LS
MR	0.13	1.95×	2.35×	2.38×	2.28×	2.21×	2.38×	**2.52**×
CT	0.50	2.05×	2.24×	1.67×	2.15×	1.96×	1.77×	**2.68**×
CR	7.18	3.24×	3.69×	3.81×	3.70×	3.71×	3.77×	**3.96**×
XR	10.1	1.43×	1.74×	**1.76**×	1.75×	**1.76**×	1.67×	**1.76**×
MG1	9.35	7.20×	8.79×	8.67×	8.84×	8.87×	8.25×	**8.91**×
MG2	9.35	7.21×	8.77×	8.65×	8.83×	8.85×	8.24×	**8.90**×
MG3	27.3	2.04×	2.24×	2.32×	2.31×	2.34×	2.22×	**2.38**×
MG4	26.0	3.10×	3.47×	3.59×	3.59×	3.62×	3.51×	**3.71**×
CT1	0.50	2.47×	2.79×	2.49×	2.54×	2.29×	2.70×	**3.19**×
CT2	0.50	3.24×	3.49×	2.87×	3.11×	2.72×	3.29×	**4.54**×
MG-N	27.3	2.04×	2.24×	2.32×	2.31×	2.34×	2.23×	**2.38**×
MR1	0.50	1.79×	2.09×	2.14×	2.10×	2.08×	2.13×	**2.30**×
MR2	2.00	2.77×	3.28×	3.34×	3.31×	3.31×	3.35×	**3.52**×
MR3	0.50	3.47×	3.93×	4.09×	3.89×	3.84×	4.33×	**4.51**×
MR4	0.50	3.58×	4.12×	4.18×	4.09×	4.03×	4.21×	**4.49**×
NM1	0.50	4.88×	5.15×	5.02×	5.26×	5.28×	5.76×	**6.28**×
RG1	6.86	1.45×	1.70×	1.70×	1.70×	1.69×	1.63×	**1.72**×
RG2	7.18	3.68×	4.23×	4.32×	4.28×	4.30×	4.32×	**4.51**×
RG3	5.91	5.46×	6.08×	6.82×	6.11×	6.12×	6.99×	**7.31**×
SC1	9.71	3.46×	3.71×	3.70×	3.73×	3.74×	3.85×	**4.73**×
XA1	2.00	4.37×	5.01×	4.94×	5.04×	5.03×	4.88×	**5.39**×

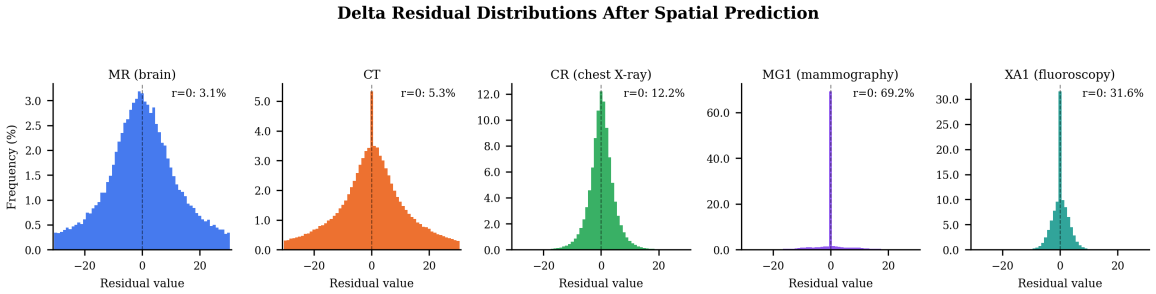

Fig. 5: Delta-residual distributions ($|r| \leq 30$ shown) after avg spatial prediction. MG1's extreme concentration (69.2% at zero) explains its 8.79× compression ratio.

TABLE IX: Compression ratio summary—geometric means and win counts.

Codec	Geo. Mean	Best on
JPEG-LS	**3.44**×	21/21
Wavelet V2 SIMD	3.28×	0/21
HTJ2K	3.15×	0/21
MIC (Delta+RLE+FSE)	3.12×	0/21
Delta+Zstd-19	3.03×	0/21

C. Lightweight Client-Side Decoding

MIC ships two browser decoder implementations: a pure JS ES module (∼20 KB, zero dependencies) and a Go WASM build (∼2.5 MB). A web-based DICOM viewer can fetch a `.mic` file directly from object storage and decode it client-side, eliminating server-side transcoding. As of the submission date, no production-ready WebAssembly or JavaScript decoder exists for HTJ2K [33] or JPEG-LS [32].

D. Pathway to Clinical Impact

MIC could be registered as a DICOM Private Transfer Syntax, allowing PACS vendors to adopt it without modifying the DICOM standard. The ∼20 KB JavaScript decoder enables a direct storage-and-serve distribution model that bypasses server-side transcoding entirely.

VIII. Conclusion

This paper presented MIC, a lossless medical image compression pipeline based on spatial prediction, 16-bit-native run-length representation, and large-alphabet table-based entropy coding. The key results across 21 clinical DICOM images spanning 10 modalities:

- **Compression ratios**: 1.7×–8.9× (grayscale), 1.56×–6.24× (single-frame RGB).
- **vs. Delta+Zstandard**: 5–22% better compression on all 21 images (geometric mean +14%), attributable to the structural cost of byte-splitting 16-bit residuals.
- **vs. HTJ2K**: MIC-4state-C exceeds HTJ2K decompression on 19/21 images single-threaded (ARM64) and 16/21 (AMD64); PICS-C-8 exceeds HTJ2K on all 21 images on both platforms.
- **vs. JPEG-LS**: 1.7–5.0× faster decompression; JPEG-LS retains the best lossless ratios (geometric mean 3.44× vs. MIC's 3.12×).
- **Multi-state decoder**: geomean +68% isolated FSE speedup (ARM64; +43% AMD64) with no ratio penalty.
- **Browser decoder**: ∼20 KB pure JS decoder; PICS + `worker_threads` achieves 483 MB/s (MG1 in Node.js).

TABLE X: Decompression throughput (MB/s), Apple M2 Max (ARM64). Bold = fastest per row (single-threaded: first 5 cols; PICS: last 3 cols).

Image	MIC-Go	MIC-4s-C	Wav+SIMD	HTJ2K	JPEG-LS	PICS-C-2	PICS-C-4	PICS-C-8
MR	144	**348**	248	265	102	530	**710**†	482
CT	191	**356**	316	307	137	524	955	**1,092**
CR	296	524	**567**	367	153	867	1,635	**2,661**
XR	308	533	**627**	334	108	874	1,666	**3,025**
MG1	482	683	678	**810**	409	1,205	2,112	**3,656**
MG2	479	686	697	**790**	416	1,225	2,120	**3,773**
MG3	308	**531**	422	338	153	864	1,673	**3,117**
MG4	417	**625**	516	548	184	1,093	2,004	**3,689**
CT1	239	**436**	425	362	182	686	1,013	**1,183**
CT2	238	439	**481**	375	175	676	1,041	**1,189**
MG-N	316	**536**	468	340	153	883	1,711	**3,175**
MR1	278	**521**	435	325	116	751	1,207	**1,402**
MR2	333	**563**	498	388	172	913	1,552	**2,466**
MR3	375	**639**	507	441	236	908	1,430	**1,614**
MR4	316	**571**	479	406	197	818	1,341	**1,558**
NM1	327	**632**	575	410	210	888	1,400	**1,679**
RG1	235	406	**584**	332	104	602	1,128	**2,017**
RG2	367	590	**644**	443	193	986	1,803	**3,194**
RG3	374	604	**656**	562	246	1,035	1,944	**3,302**
SC1	375	**587**	388	401	229	1,017	1,861	**3,279**
XA1	331	**576**	459	419	204	928	1,583	**2,493**

TABLE XI: Decompression throughput (MB/s), Intel Core Ultra 9 285K (AMD64). Bold = fastest per row within single-threaded and PICS groups.

Image	MIC-Go	MIC-4s-C	MIC-4s-SIMD	Wav+SIMD	HTJ2K	PICS-C-4	PICS-C-8
MR	251	501	**708**	194	570	301	124†
CT	231	403	487	364	**544**	**676**	339
CR	324	599	**744**	723	708	1,738	**2,435**
XR	363	601	**803**	681	570	1,714	**1,994**
MG1	617	789	1,119	746	**1,235**	1,872	**2,514**
MG2	562	800	1,166	848	**1,297**	**2,244**	2,085
MG3	357	633	669	**678**	644	1,823	**2,538**
MG4	523	743	773	**808**	916	2,124	**2,707**
CT1	322	520	**676**	525	657	**857**	423
CT2	293	514	636	**705**	627	**847**	687
MG-N	368	635	669	**705**	643	1,872	**2,191**
MR1	356	609	**766**	532	654	**945**	366
MR2	352	658	**975**	601	749	**1,121**	793
MR3	450	728	**809**	676	802	**920**	705
MR4	390	596	**861**	738	660	493	**701**
NM1	367	**717**	668	715	627	**783**	405
RG1	302	497	593	**705**	557	1,248	**1,494**
RG2	433	676	**861**	811	823	1,841	**1,912**
RG3	460	706	858	881	**894**	**1,969**	1,613
SC1	464	695	**888**	710	728	1,819	**1,889**
XA1	413	693	**836**	538	797	1,173	**1,513**

TABLE XII: Encoding throughput (MB/s), Apple M2 Max (ARM64), selected images. Bold = fastest per row.

Image	MIC-Go	MIC-4s-C	MIC-C	HTJ2K	JPEG-LS	PICS-8
MR	180	243	**305**	217	104	103
CT	208	314	332	258	166	**358**
CR	307	441	416	311	119	**1,195**
XR	336	498	500	269	123	**1,136**
MG1	512	621	651	676	320	**2,001**
MG2	517	644	702	700	315	**2,095**
MG3	355	469	425	293	153	**1,491**
MG4	458	554	503	451	234	**1,984**

TABLE XIII: JavaScript decoder throughput (4-state), Node.js v24.8, Apple M2 Max. Median over 20 iterations.

Image	Dimensions	Ratio	ms	MB/s
MR	256×256	2.35×	3.0	42
CT	512×512	2.24×	13.5	37
CR	2140×1760	3.69×	161.2	45
MG1	2457×1996	8.79×	80.9	116
MG3	4774×3064	2.29×	638.4	44

TABLE XIV: PICS parallel JavaScript decoder (`worker_threads`), Apple M2 Max. Speedup relative to 1 worker.

Image	Strips	ms	MB/s	Speedup
MR	4	1.3	94	2.57×
CT	4	5.0	101	2.95×
CR	8	31.7	227	5.53×
MG1	8	19.4	483	4.36×

TABLE XV: Ratio vs. decompression throughput (geometric means) and decoder source size.

Codec	Ratio	Decomp.	LOC	Browser
Δ+RLE+FSE (Go)	3.12×	310	1.1k	**Yes**
Δ+RLE+FSE (4s-C)	3.12×	530	1.5k	**Yes**
HTJ2K (OpenJPH)	3.15×	460	20k	No
JPEG-LS (CharLS)	3.44×	130	5.0k	No

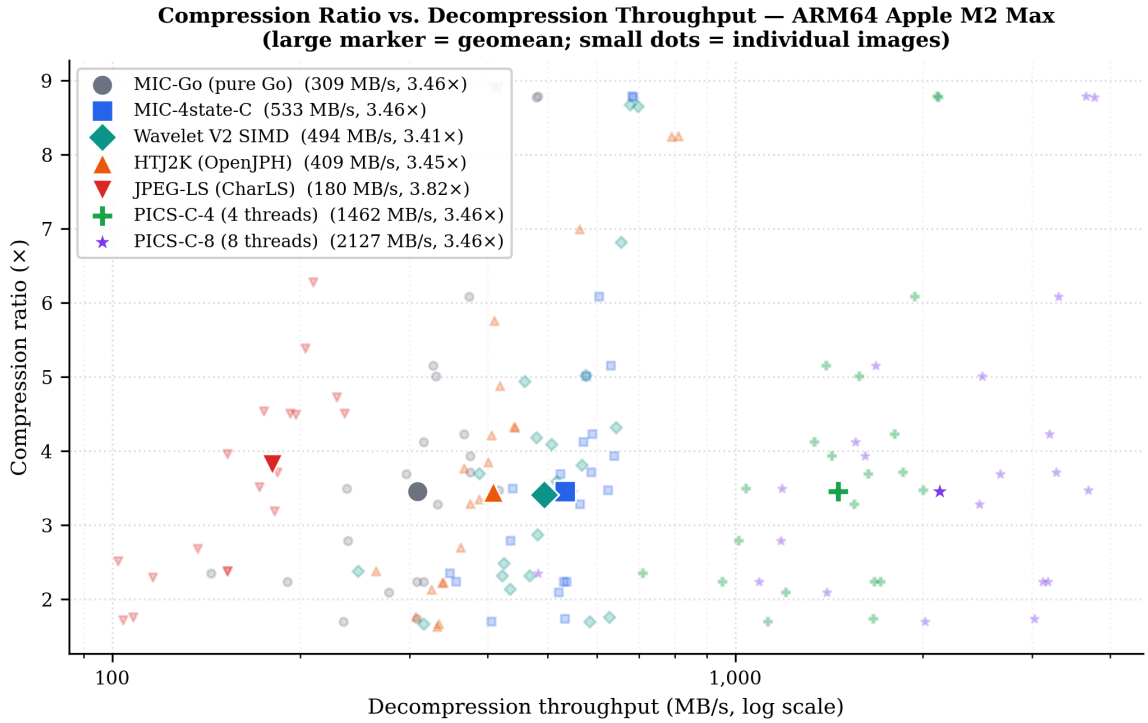

Fig. 6: Compression ratio vs. decompression throughput (ARM64). Large markers = geometric means; small dots = individual images.

- **Implementation**: ∼1,100 lines of Go code—approximately 18× less than HTJ2K.

Overall, the study indicates that 16-bit-native entropy coding is a useful design point for lossless medical image compression, particularly in applications where decompression throughput, implementation simplicity, and deployment portability are important. Future work should strengthen the evidence base with larger datasets, shuffle-based general-purpose baselines, formal statistical reporting, and direct browser benchmarking.

MIC is open-source and available at https://github.com/pappuks/medical-image-codec.

The two tables in this appendix report per-image *encoding* throughput (MB/s of uncompressed pixel data processed) across all codec variants, on the two evaluation platforms. They complement the decompression-focused results in Section VI: Table XII in the main text shows ARM64 encoding for a selected subset; the tables below give the full 21-image sweeps. Appendix A covers the Intel Core Ultra 9 285K (AMD64) platform, and Appendix B covers the Apple M2 Max (ARM64) platform. Rows are individual DICOM test images and columns are codec variants (MIC-Go, MIC-4s, MIC-4s-C, MIC-C, Wavelet+SIMD, HTJ2K, JPEG-LS, and the parallel-strip PICS-2/4/8 configurations). Bold entries mark the fastest encoder on each row.

Appendix A
AMD64 Encoding Throughput

Appendix B
ARM64 Encoding Throughput (Full Table)

References

[1] D. S. Taubman and M. W. Marcellin, *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Springer, 2002.

[2] D. S. Taubman, "High throughput JPEG 2000 (HTJ2K): Algorithm, performance evaluation, and potential," *Proc. SPIE*, vol. 11137, 2019.

[3] M. J. Weinberger, G. Seroussi, and G. Sapiro, "The LOCO-I lossless image compression algorithm: Principles and standardization into JPEG-LS," *IEEE Trans. Image Process.*, vol. 9, no. 8, pp. 1309–1324, 2000.

[4] J. Duda, "Asymmetric numeral systems," arXiv:0902.0271, 2009.

[5] J. Duda, "Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding," arXiv:1311.2540, 2013.

[6] Y. Collet, "Finite State Entropy—A new breed of entropy coder," https://github.com/Cyan4973/FiniteStateEntropy, 2013.

[7] Y. Collet and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," RFC 8878, IETF, Feb. 2021.

[8] S. Mahapatra and K. Singh, "An FPGA-based implementation of multi-alphabet arithmetic coding," *IEEE Trans. Circuits Syst. I*, vol. 54, no. 8, pp. 1678–1686, 2007.

[9] D. Kosolobov, "Efficiency of ANS Entropy Encoders," arXiv:2201.02514, 2022.

TABLE XVI: Encoding throughput (MB/s), Intel Core Ultra 9 285K (AMD64). Bold = fastest per row.

Image	MIC-Go	MIC-4s-C	MIC-C	Wav+SIMD	HTJ2K	JPEG-LS	PICS-4	PICS-8
MR	121	**290**	273	85	177	71	186	128
CT	180	**359**	311	84	178	104	301	312
CR	233	461	423	120	193	89	732	**1,102**
XR	254	550	519	127	214	95	775	**1,212**
MG1	380	861	820	155	508	239	1,112	**1,651**
MG2	380	857	830	153	507	235	1,119	**1,676**
MG3	256	556	514	97	202	109	832	**1,317**
MG4	336	738	710	107	354	162	1,098	**1,901**
CT1	206	**413**	384	94	216	132	310	329
CT2	192	**371**	340	89	194	132	320	294
MG-N	254	562	529	99	202	107	842	**1,353**
MR1	221	**460**	429	101	195	92	364	347
MR2	275	566	532	107	230	102	643	**857**
MR3	300	**609**	591	119	292	142	498	448
MR4	249	**494**	451	108	226	142	370	368
NM1	237	**467**	444	117	242	132	360	302
RG1	229	485	397	123	198	70	651	**942**
RG2	280	582	488	125	254	112	842	**1,220**
RG3	275	550	512	128	273	143	863	**1,222**
SC1	286	577	551	83	235	169	924	**1,425**
XA1	242	496	471	101	219	114	616	**832**

TABLE XVII: Encoding throughput (MB/s), Apple M2 Max (12-core ARM64), all variants. Bold = fastest per row.

Image	MIC-Go	MIC-4s	MIC-4s-C	MIC-C	Wav+SIMD	HTJ2K	JPEG-LS	PICS-2	PICS-4	PICS-8
MR	180	219	243	**305**	102	217	104	111	104	103
CT	208	222	314	332	89	258	166	195	**358**	313
CR	307	312	441	416	138	311	119	488	856	**1,195**
XR	336	322	498	500	126	269	123	411	738	**1,136**
MG1	512	508	621	651	162	676	320	858	1,437	**2,001**
MG2	517	505	644	702	160	700	315	829	1,372	**2,095**
MG3	355	352	469	425	95	293	153	530	862	**1,491**
MG4	458	463	554	503	106	451	234	654	1,186	**1,984**
CT1	292	267	383	388	89	314	204	299	**397**	347
CT2	228	236	356	335	95	294	231	286	**373**	343
MG-N	341	357	424	427	94	309	156	538	929	**1,421**
MR1	309	307	400	**441**	104	271	151	176	301	301
MR2	350	350	498	514	106	339	136	463	672	**821**
MR3	385	379	538	**568**	121	389	183	287	434	516
MR4	305	310	466	**471**	118	354	252	262	363	325
NM1	302	302	438	444	114	346	178	273	323	**474**
RG1	284	315	471	367	127	256	107	349	648	**985**
RG2	390	398	493	520	140	402	155	598	1,034	**1,583**
RG3	374	388	488	485	153	455	198	566	1,029	**1,389**
SC1	414	413	508	466	97	349	297	655	1,170	**1,712**
XA1	335	324	449	463	111	357	154	437	708	**944**

[10] R. Bamler, "Understanding Entropy Coding With Asymmetric Numeral Systems (ANS): a Statistician's Perspective," arXiv:2201.01741, 2022.

[11] F. Giesen, "Interleaved entropy coders," arXiv:1402.3392, 2014.

[12] F. Giesen, "ryg_rans: Public domain rANS encoder/decoder," https://github.com/rygorous/ryg_rans, 2014.

[13] F. Lin, K. Arunruangsirilert, H. Sun, and J. Katto, "Recoil: Parallel rANS Decoding with Decoder-Adaptive Scalability," *Proc. 52nd ICPP '23*, pp. 31–40, 2023.

[14] A. Weissenberger and B. Schmidt, "Massively Parallel ANS Decoding on GPUs," *Proc. 48th ICPP '19*, Article 100, 2019.

[15] H. S. Malvar and G. J. Sullivan, "YCoCg-R: A color space with RGB reversibility and low dynamic range," JVT Doc. JVT-I014, 2003.

[16] W3C, "Portable Network Graphics (PNG) Specification (Second Edition)," ISO/IEC 15948:2003, Nov. 2003.

[17] A. N. Aous, "OpenJPH: An open-source implementation of High-Throughput JPEG 2000," https://github.com/aous72/OpenJPH, 2024.

[18] Team CharLS, "CharLS: A C/C++ JPEG-LS library implementation," https://github.com/team-charls/charls, 2024.

[19] As of the submission date, no production-ready WebAssembly or JavaScript decoder for High-Throughput JPEG 2000 was identified. The OpenJPH open-source library does not provide a WASM build or npm package.